Software Technology 2: Week 6 Tutorial & Computer Lab

Task 1: Linked Lists, Stacks, Queues, and Priority Queues

Part a) Testing & Benchmarking the simple python lists, single linked and double linked list data structure

Run the script linked_lists_practice.py for testing the simple python lists, single linked and double linked list data structure code and benchmark their performance for different data sizes. Report on the performance (screenshots)for each data size. Currently it is set for 1000. Test it for 10000 and 50000.

1000 Elements

```
Testing with 1000 elements, search and delete value: 38949
Python list insert at beginning: 0.000001 s
Python list search: 0.000006 s
Python list delete: 0.000015 s
Singly Linked List insert at beginning: 0.000001 s
Singly Linked List search: 0.000021 s
Singly Linked List delete: 0.000041 s
Doubly Linked List insert at beginning: 0.000002 s
Doubly Linked List search: 0.000023 s
Doubly Linked List delete: 0.000036 s
```

10000 Elements

```
└─ python linked_lists_practice.py
Testing with 10000 elements, search and delete value: 6333
Python list insert at beginning: 0.000004 s
Python list search: 0.000056 s
Python list delete: 0.000123 s
Singly Linked List insert at beginning: 0.000005 s
Singly Linked List search: 0.000182 s
Singly Linked List delete: 0.000399 s
Doubly Linked List insert at beginning: 0.000002 s
Doubly Linked List search: 0.000183 s
Doubly Linked List delete: 0.000336 s
```

50000 Elements

```
└─ python linked_lists_practice.py
Testing with 50000 elements, search and delete value: 808266
Python list insert at beginning: 0.000022 s
Python list search: 0.000269 s
Python list delete: 0.000642 s
Singly Linked List insert at beginning: 0.000002 s
Singly Linked List search: 0.000749 s
Singly Linked List delete: 0.001591 s
^[[57429u^[[57429uDoubly Linked List insert at beginning: 0.000002 s
Doubly Linked List search: 0.000626 s
Doubly Linked List delete: 0.001146 s
```

Part b) Testing & Benchmarking the stack, queue and priority queue datastructure code and double linked list

1000 values

```
└─ python stacks_queues_practice.py
Benchmarking with 1000 values
Stack (list) Push: 0.000027 s
Stack (list) Peek: 0.000001 s
Stack (list) Pop: 0.000052 s
Queue (deque) Enqueue: 0.000037 s
Queue (deque) Peek: 0.000000 s
Queue (deque) Dequeue: 0.000055 s
Priority Queue (heapq) Push: 0.000090 s
Priority Queue (heapq) Peek: 0.000000 s
Priority Queue (heapq) Pop: 0.000345 s
```

10000 Values

```
└─ python stacks_queues_practice.py
Benchmarking with 10000 values
Stack (list) Push: 0.000350 s
Stack (list) Peek: 0.000001 s
Stack (list) Pop: 0.000575 s
Queue (deque) Enqueue: 0.000347 s
Queue (deque) Peek: 0.000001 s
Queue (deque) Dequeue: 0.000574 s
Priority Queue (heapq) Push: 0.000983 s
Priority Queue (heapq) Peek: 0.000001 s
Priority Queue (heapq) Pop: 0.004873 s
```

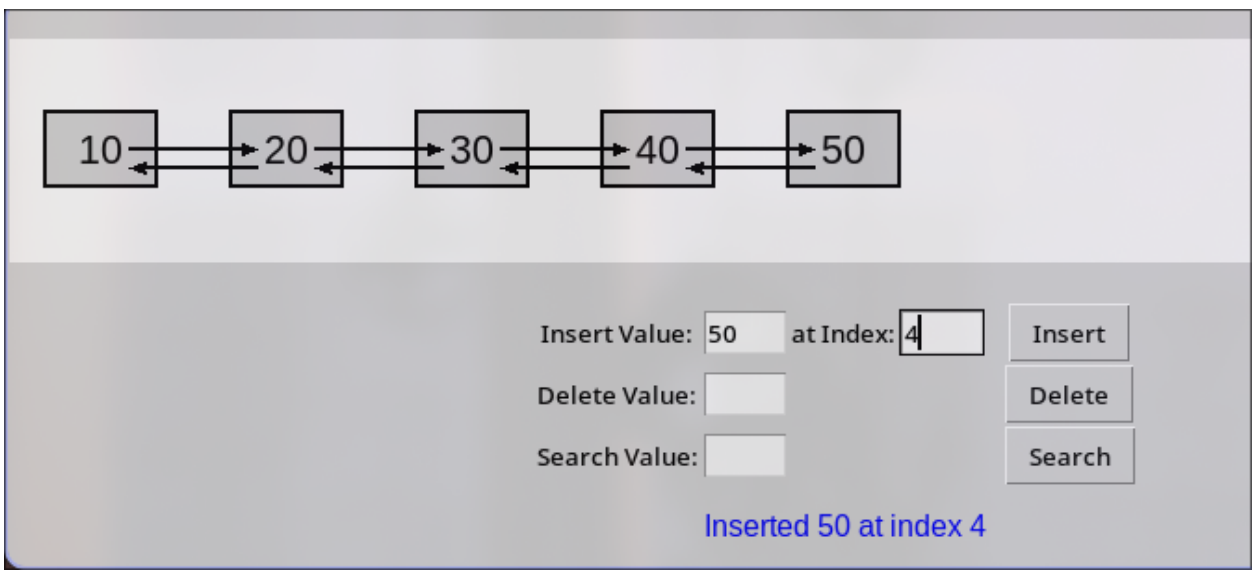
100000 Values

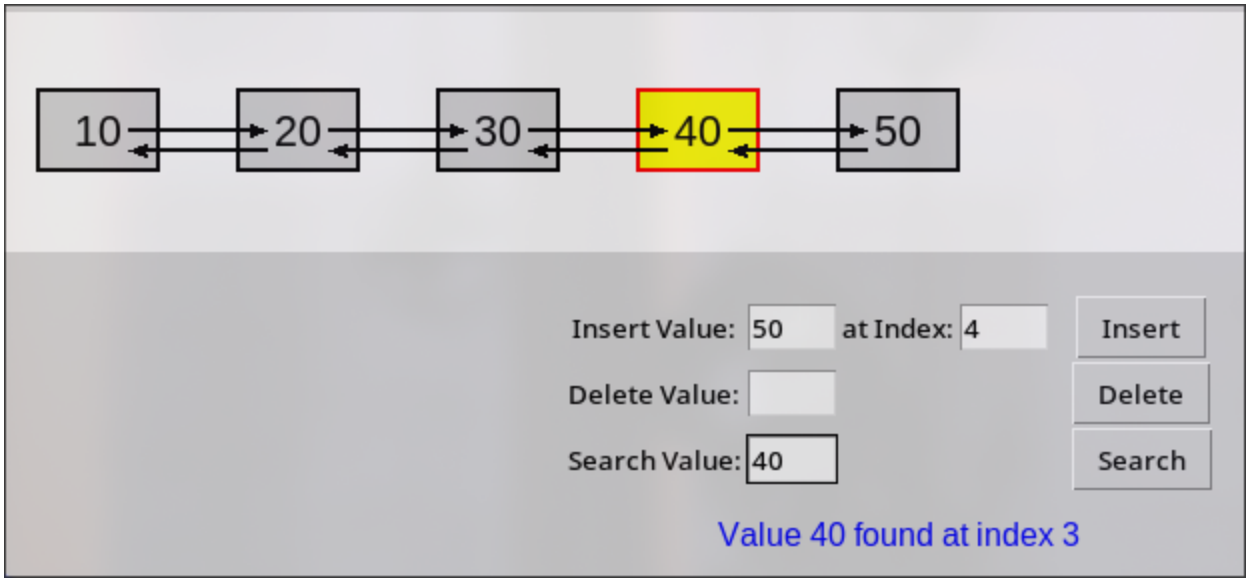
```
└─ python stacks_queues_practice.py
Benchmarking with 100000 values
Stack (list) Push: 0.002521 s
Stack (list) Peek: 0.000001 s
Stack (list) Pop: 0.006160 s
Queue (deque) Enqueue: 0.003397 s
Queue (deque) Peek: 0.000001 s
Queue (deque) Dequeue: 0.005851 s
Priority Queue (heapq) Push: 0.009245 s
Priority Queue (heapq) Peek: 0.000001 s
Priority Queue (heapq) Pop: 0.080562 s
```

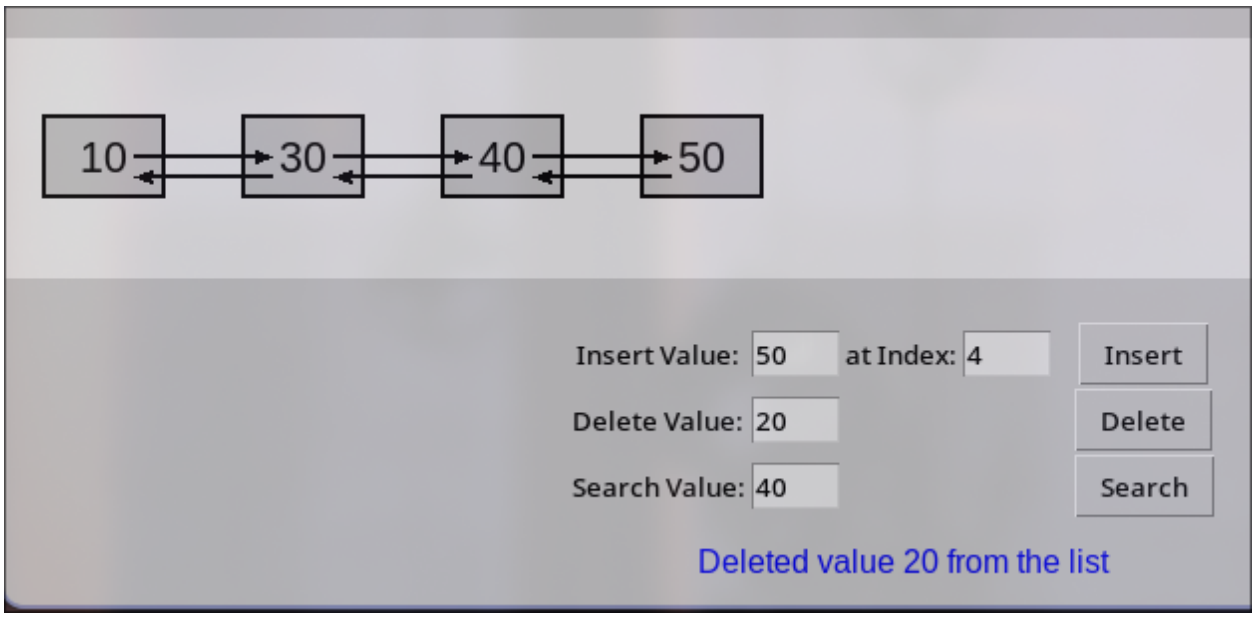
Stack - O(1)
Queue - O(1)
Priority Queue - O(log n)

Task 2: Visualising Linked Lists

Testing Insertion, Searching and Deleting Values







Modify the visualiser code for this task to allow visualisation of a single linked list.

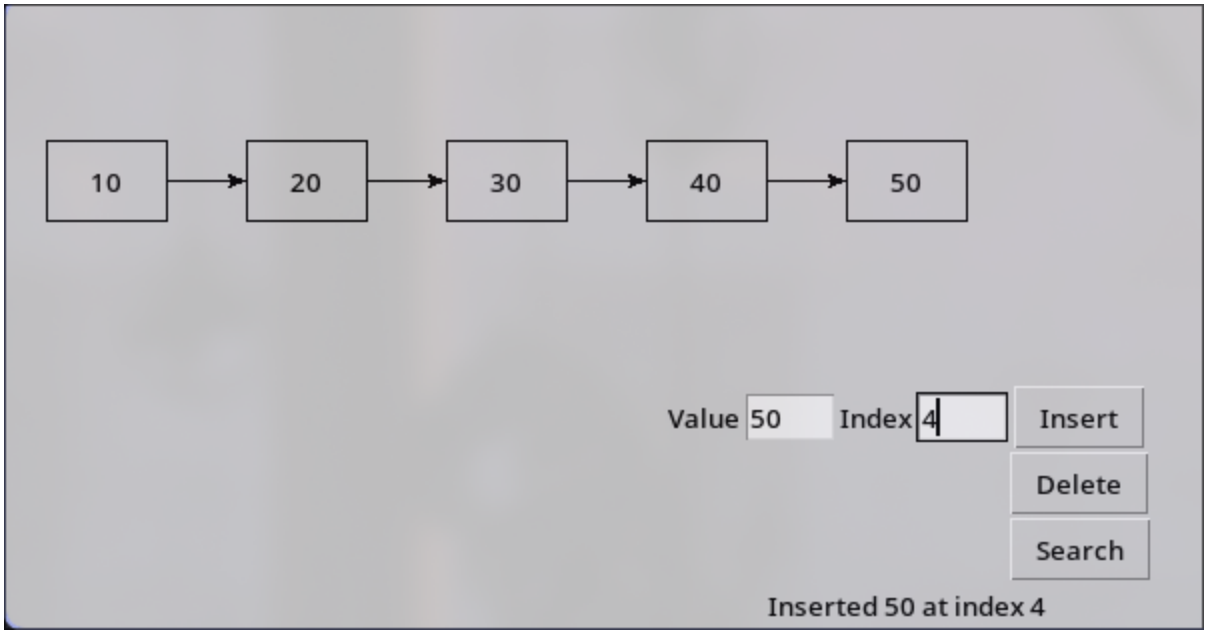
```
import tkinter as tk
from tkinter import messagebox

NODE_WIDTH = 60
NODE_HEIGHT = 40
HORIZONTAL_GAP = 40

class Node:
```

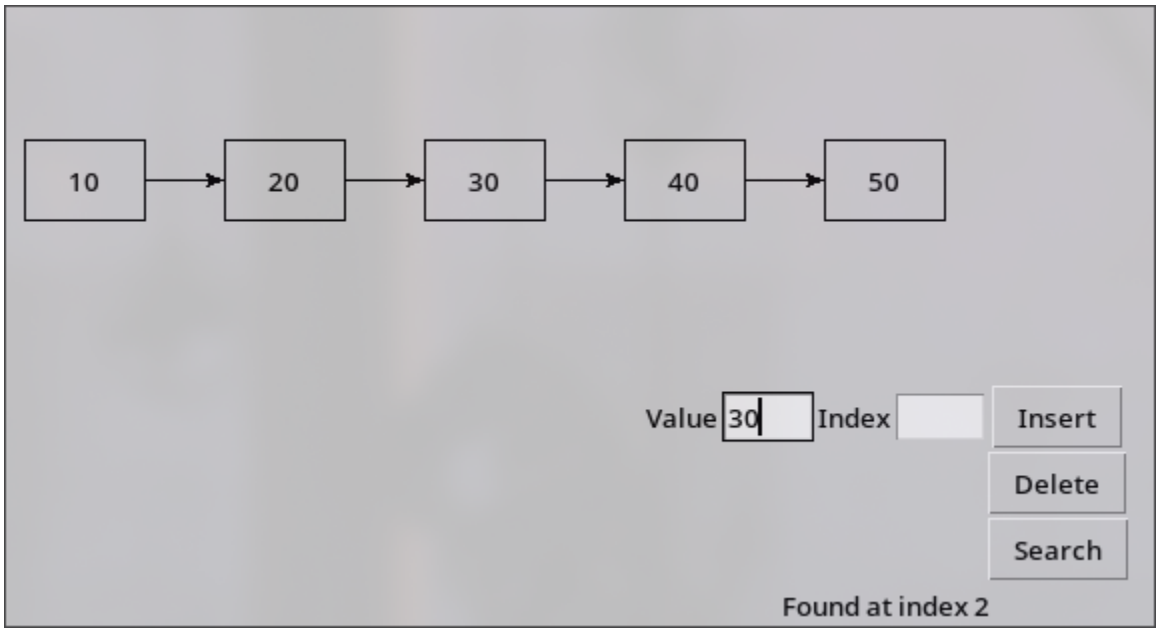
```python
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_index(self, data, index):
        new = Node(data)

        if index == 0:
            new.next = self.head
            self.head = new
            return True

        current = self.head
        count = 0

        while current and count < index - 1:
            current = current.next
            count += 1

        if current is None:
            return False

        new.next = current.next
        current.next = new
        return True

    def delete_value(self, val):
        current = self.head
        prev = None

        while current:
            if current.data == val:
                if prev:
                    prev.next = current.next
                else:
```

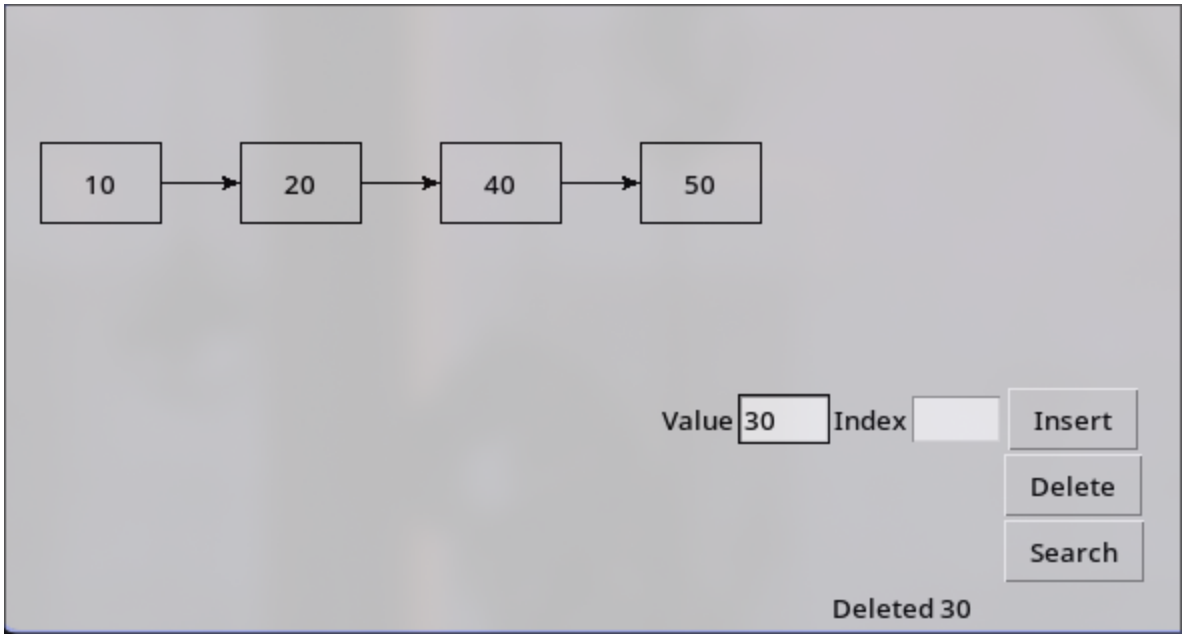
```
                    self.head = current.next
                return True
            prev = current
            current = current.next
        return False

    def search(self, val):
        current = self.head
        index = 0

        while current:
            if current.data == val:
                return index
            current = current.next
            index += 1

        return -1

    def to_list(self):
        arr = []
        cur = self.head
        while cur:
            arr.append(cur.data)
            cur = cur.next
        return arr


class Visualizer:
    def __init__(self, root):
        self.root = root
        self.root.title("Singly Linked List Visualizer")

        self.list = SinglyLinkedList()

        self.canvas = tk.Canvas(root, width=900, height=150)
        self.canvas.pack(pady=20)

        frame = tk.Frame(root)
        frame.pack()
```

```
        tk.Label(frame, text="Value").grid(row=0, column=0)
        self.val_entry = tk.Entry(frame, width=5)
        self.val_entry.grid(row=0, column=1)

        tk.Label(frame, text="Index").grid(row=0, column=2)
        self.idx_entry = tk.Entry(frame, width=5)
        self.idx_entry.grid(row=0, column=3)

        tk.Button(frame, text="Insert", command=self.insert).grid(row=0,
column=4)
        tk.Button(frame, text="Delete", command=self.delete).grid(row=1,
column=4)
        tk.Button(frame, text="Search", command=self.search).grid(row=2,
column=4)

        self.status = tk.Label(root, text="")
        self.status.pack()

        self.draw()

    def draw(self):
        self.canvas.delete("all")
        x = 20
        y = 50

        nodes = self.list.to_list()

        for i, val in enumerate(nodes):
            self.canvas.create_rectangle(x, y, x+NODE_WIDTH, y+NODE_HEIGHT)
            self.canvas.create_text(x+30, y+20, text=str(val))

            if i < len(nodes) - 1:
                self.canvas.create_line(x+60, y+20, x+100, y+20,
arrow=tk.LAST)

            x += NODE_WIDTH + HORIZONTAL_GAP

    def insert(self):
        try:
            val = int(self.val_entry.get())
```

```
            idx = int(self.idx_entry.get())
        except:
            messagebox.showerror("Error", "Invalid input")
            return

        if not self.list.insert_at_index(val, idx):
            messagebox.showerror("Error", "Index out of range")
        else:
            self.status.config(text=f"Inserted {val} at index {idx}")
            self.draw()

    def delete(self):
        val = int(self.val_entry.get())
        if self.list.delete_value(val):
            self.status.config(text=f"Deleted {val}")
            self.draw()
        else:
            messagebox.showerror("Error", "Value not found")

    def search(self):
        val = int(self.val_entry.get())
        idx = self.list.search(val)

        if idx != -1:
            self.status.config(text=f"Found at index {idx}")
        else:
            self.status.config(text="Not found")


root = tk.Tk()
Visualizer(root)
root.mainloop()
```

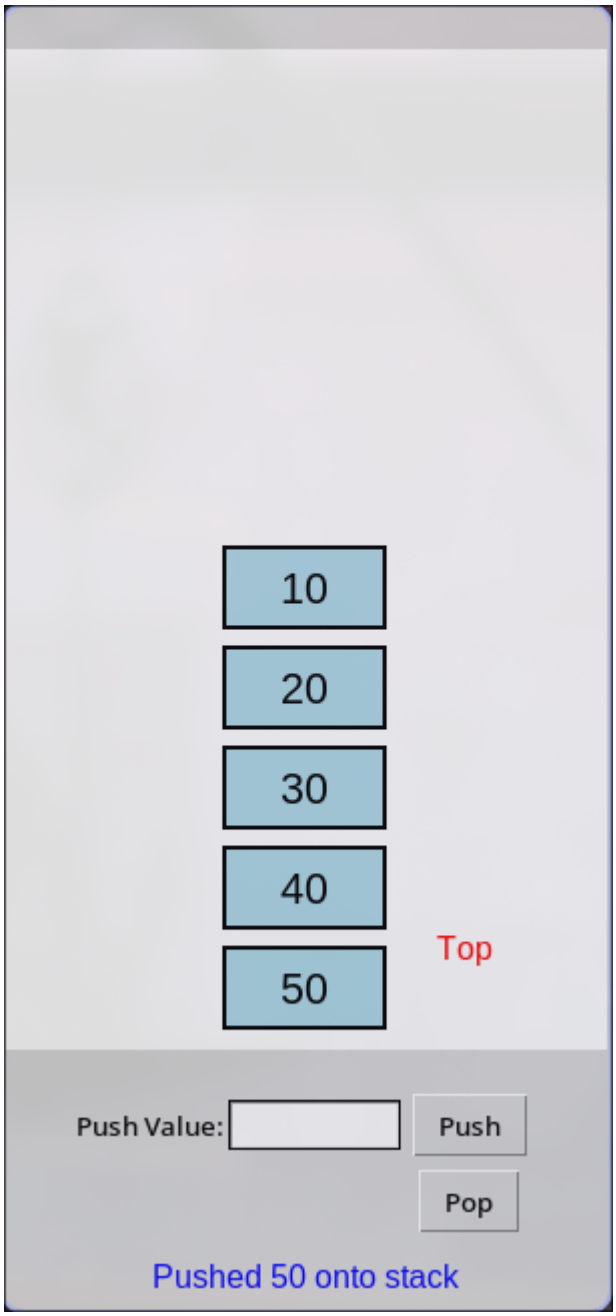

10
20
30
40
50
Value
50
Index
4
Insert
Delete
Search
Inserted 50 at index 4

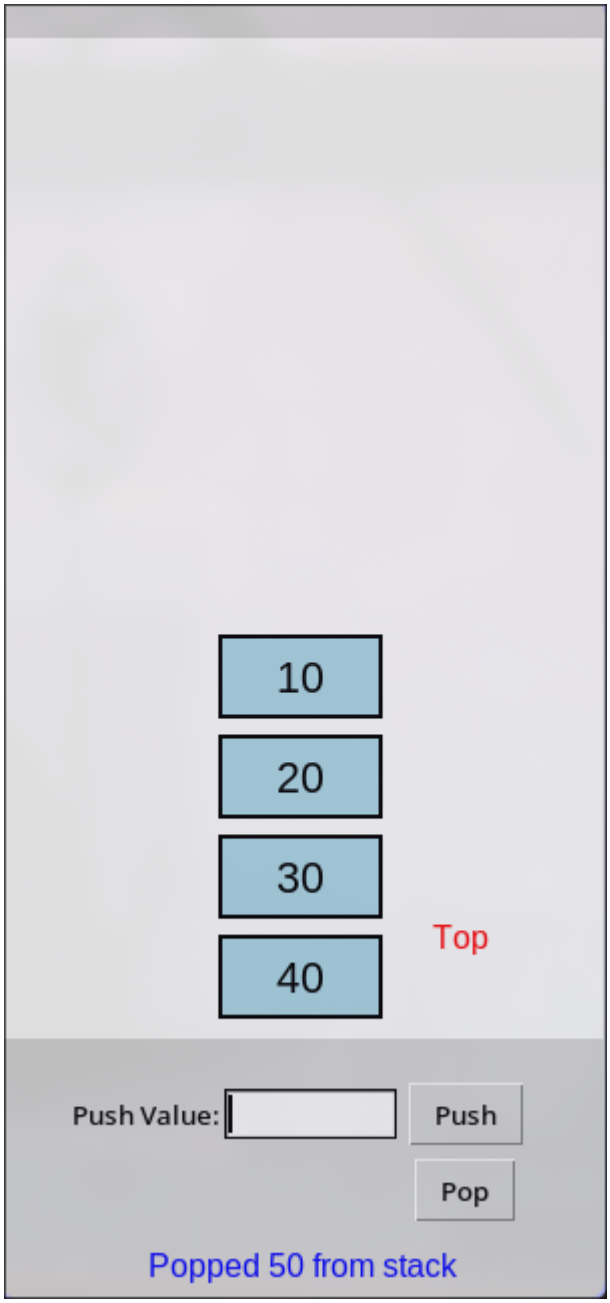

10
20
30
40
50
Value
30
Index
Insert
Delete
Search
Found at index 2


10
20
40
50
Value
30
Index
Insert
Delete
Search
Deleted 30

Task 3: Visualising Stacks and Queues

Testing Pushing and Popping Values





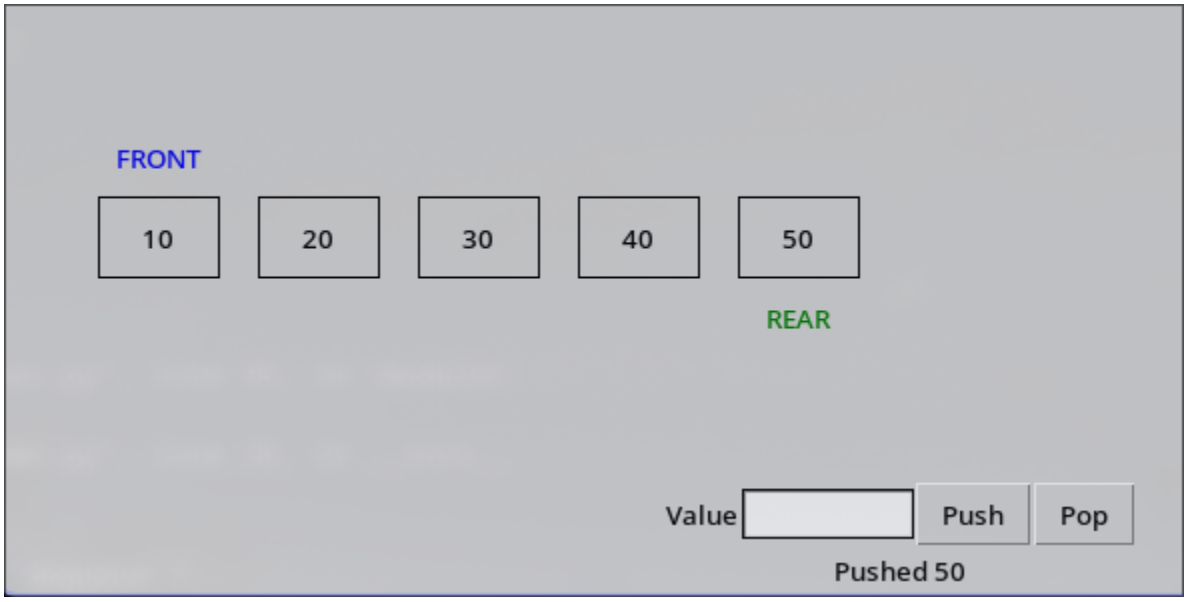
Modify the visualiser code for this task to allow visualisation of queue .

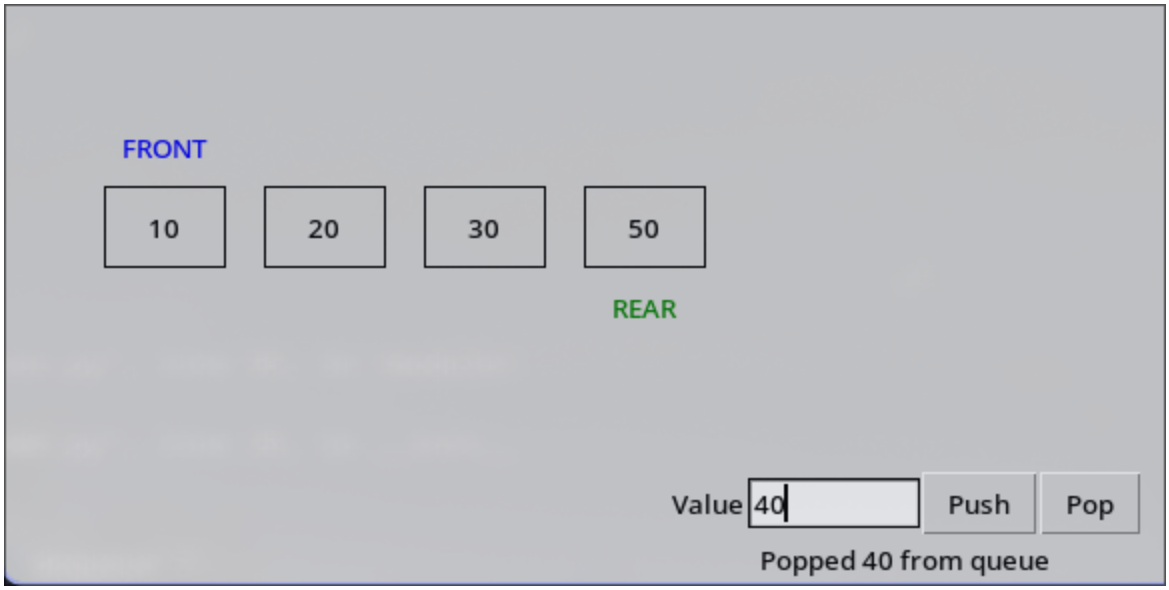
```
import tkinter as tk
from tkinter import messagebox
```

```
NODE_WIDTH = 60
NODE_HEIGHT = 40
HORIZONTAL_GAP = 20


class QueueVisualizer:
   def __init__(self, root):
       self.root = root
       self.root.title("Queue Visualizer (FIFO)")

       self.queue = []

       # Canvas
       self.canvas = tk.Canvas(root, width=900, height=200)
       self.canvas.pack(pady=20)

       # Controls
       frame = tk.Frame(root)
       frame.pack()

       tk.Label(frame, text="Value").grid(row=0, column=0)
       self.entry = tk.Entry(frame, width=10)
       self.entry.grid(row=0, column=1)

       tk.Button(frame, text="Push", command=self.enqueue).grid(row=0,
column=2)
       tk.Button(frame, text="Pop", command=self.dequeue).grid(row=0,
column=3)

       # Status
       self.status = tk.Label(root, text="")
       self.status.pack()

       self.draw()

   # ----------------------------
   # DRAW QUEUE
   # ----------------------------
   def draw(self):
```

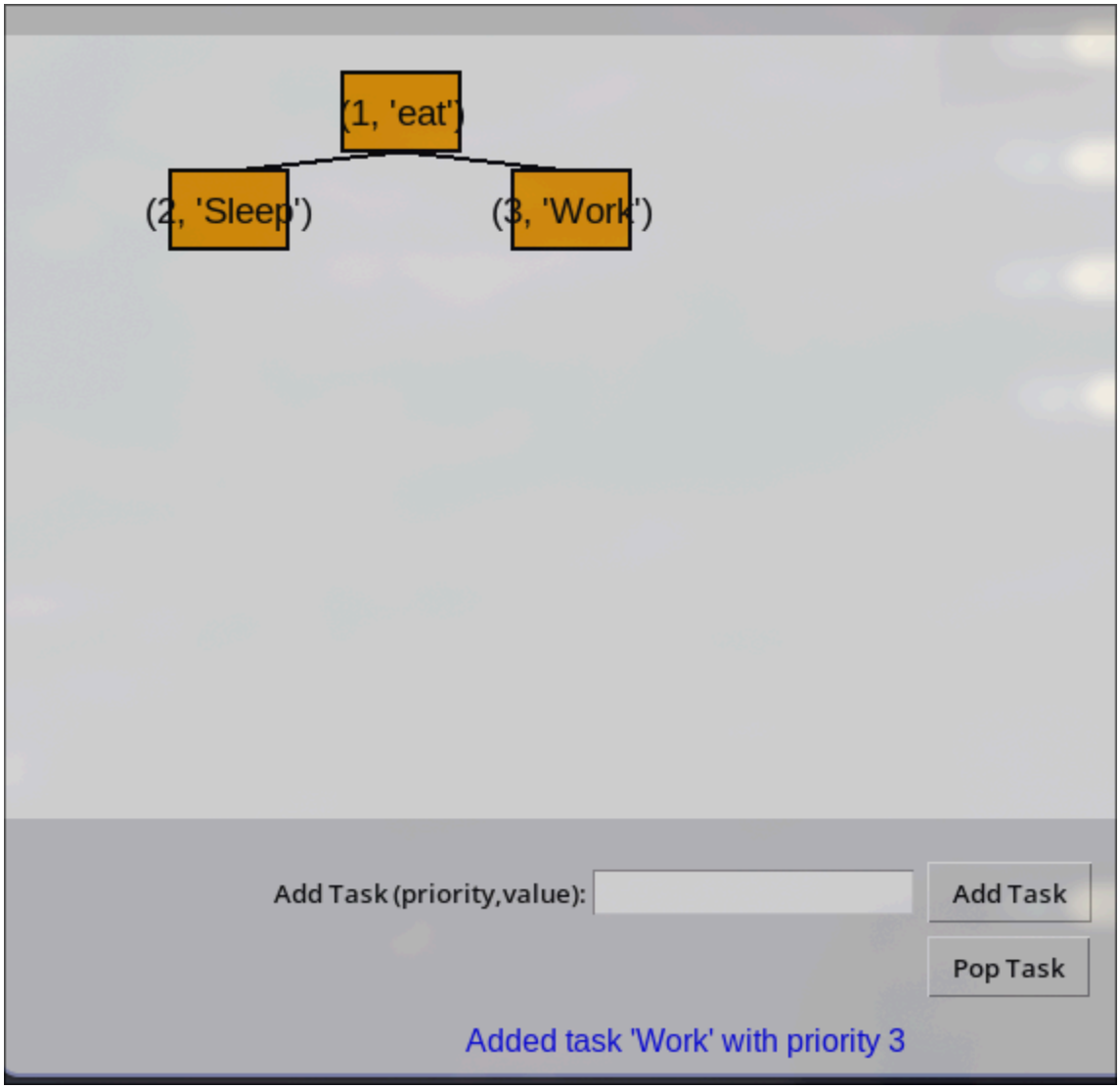
```
        self.canvas.delete("all")

        x = 50
        y = 80

        for i, val in enumerate(self.queue):
            # Draw node
            self.canvas.create_rectangle(x, y, x + NODE_WIDTH, y +
NODE_HEIGHT)
            self.canvas.create_text(x + 30, y + 20, text=str(val))

            # Labels
            if i == 0:
                self.canvas.create_text(x + 30, y - 20, text="FRONT",
fill="blue")

            if i == len(self.queue) - 1:
                self.canvas.create_text(x + 30, y + 60, text="REAR",
fill="green")

            x += NODE_WIDTH + HORIZONTAL_GAP

    # ----------------------------
    # ENQUEUE
    # ----------------------------
    def enqueue(self):
        try:
            val = int(self.entry.get())
        except:
            messagebox.showerror("Error", "Enter a valid integer")
            return

        self.queue.append(val)
        self.status.config(text=f"Pushed {val}")
        self.draw()

    # ----------------------------
    # DEQUEUE
    # ----------------------------
    def dequeue(self):
```

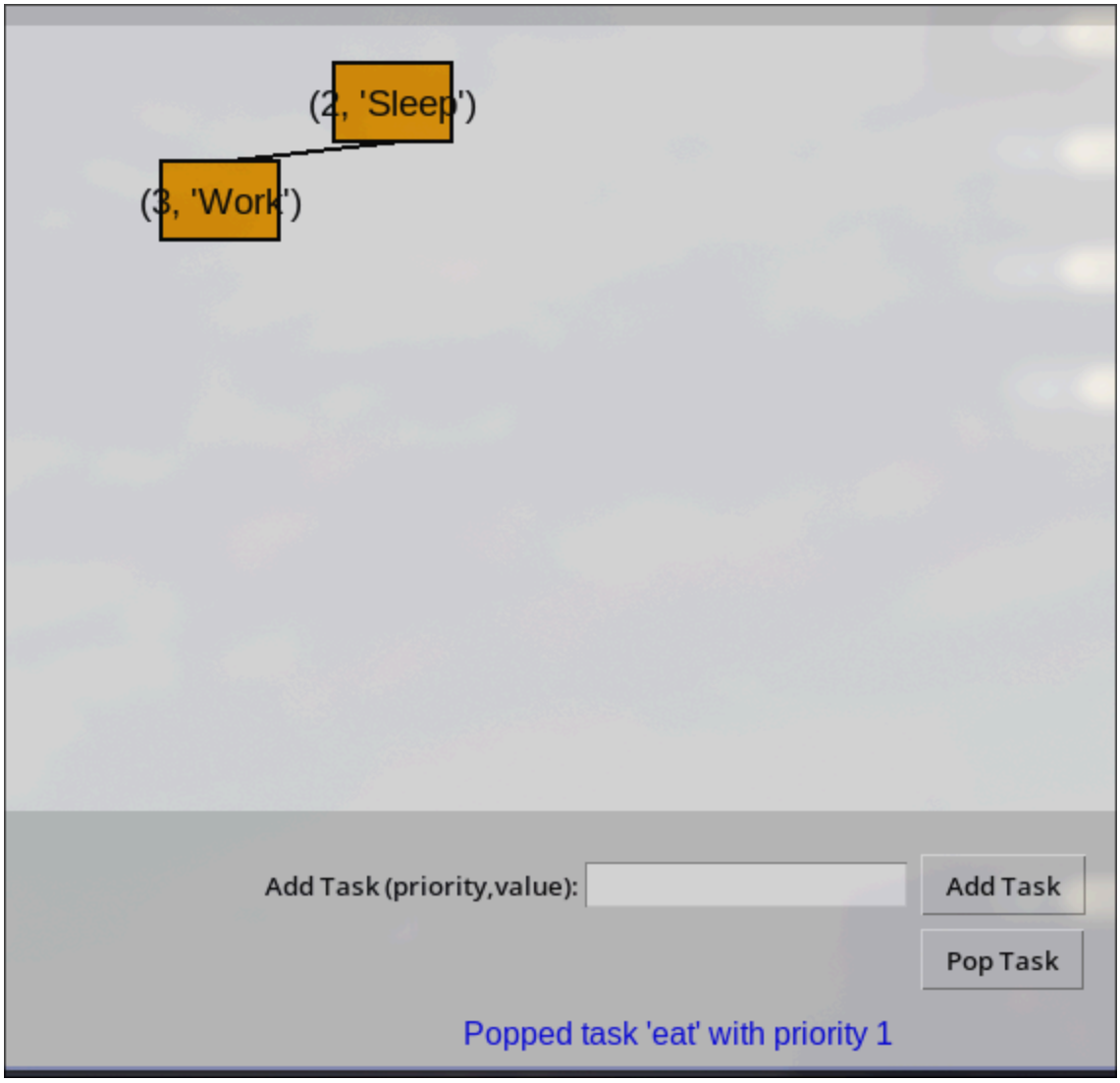
```python
        try:
            val = int(self.entry.get())
        except:
            messagebox.showerror("Error", "Enter a valid integer")
            return

        if val not in self.queue:
            messagebox.showerror("Error", "Value not found in queue")
            return

        self.queue.remove(val)
        self.status.config(text=f"Popped {val} from queue")
        self.draw()


# ----------------------------
# MAIN
# ----------------------------
root = tk.Tk()
app = QueueVisualizer(root)
root.mainloop()
```




FRONT
10
20
30
50
REAR
Value
40
Push
Pop
Popped 40 from queue

Task 4: Visualising Priority Queues




(2, 'Sleep')
(3, 'Work')
Add Task (priority,value):
Add Task
Pop Task
Popped task 'eat' with priority 1